EECE 2140

Computing Fundamentals for Engineers

System Design Overview

Autonomous Drone Fleet Simulator

A C++ Engineering Simulation Project

Team Members: **Riley Ashok**
Presthika Vijaykumar

Instructor: Dr. Fatema Nafa

Semester: Spring 2026

Repository: https://github.com/rileyashok/DroneSims

Contents

1 Project Overview

Project Description

The **Autonomous Drone Fleet Simulator** is a `C++` program that models the user creation and drone depiction of a drone show. Users are able to enter elements and shapes to create a custom drone show and then see the show visualized with early collision warning. This project makes use of object-oriented programming and data structures such as (`array`, `vector`, `struct`) to solve the real engineering problem.

1.1 Motivation

Drone shows require a large amount of precision and coordination. As drones become the next big use of technology, we require better and better ways to control them. Manual controlling is no longer sufficient, and the drones must start to become autonomous. Simulations are incredibly important for this iteration and testing to provide cheap ways to visualize without real drones and `C++` allows for precise control over memory, which will be very helpful when handling the large number of drones

1.2 Project Goals

- Implement a multi-class `C++` simulation of a customizable autonomous drone show with variable numbers of drones and mission elements.
- Have an early warning system for collision and anti-collision protocols
- Efficient and accurate shape formation, with drones calculating the shortest and safest path
- Easy coordination between fleets of drones
- Handle user input to make the show and plan the mission
- Collaborate on GitHub and LaTex for efficient development

2 Basic Functionalities

This section lists all core functionalities of the simulator together with their expected inputs, outputs, and contribution to the overall system.

2.1 Functionality Summary Table

#	Functionality	Description	Input	Output
F1	**Drone Registration**	Adds a new drone to the fleet and initializes it in the system	Drone ID, Initial Posision	Confirmation message; enables tracking and management of the drone
F2	**Mission Assignment**	Assigns a mission/shape formation to drones	Drone ID(s), mission type, target positions, drone position	Mission confirmation, allows coordinated drone tasks
F3	**Path Planning**	Calculates the optimal path for a drone	Target position, Obstacles, Drone positino	Waypoints/trajectory, ensures efficient and collision free movement.
F4	**Collision Detection / Avoidance**	Checks for potential collisions and adjusts paths	Drone positions, velocity, proximity threshold	Adjusted paths / warnings / alerts, prevents collisions and ensures safety
F5	**Simulation Update**	Updates drone positions and mission progress	Time step, drone commands	Updated positions and statuses, advances simulation loop
F6	**Visualization**	Shows drone positions graphically, simulation data, 2D/3D, helps users understand drone behavior and mission progress	Drone positions, mission plan	Graphical representation of drones, paths, and environment

2.2 Detailed Functionality Descriptions

F1 — Drone Registration

Description: Ability to add or remove a `Drone` from a fleet. It should either initialize in a intial state of 0, or allow for input of values. Each drone will have a `color` that can be set by the user.

Input: Unique `DroneID` to identify each drone, initial position (`x, y, z`) of each drone.
Output: A confirmation message for creation of drone, allow for the tracking and management of each drone. Access allowed to change drone values.
Contribution: Allows for the main drone handling functions on an individual level.

F2 — Mission Assignment

Description: The user will be allowed to input mission elements such as `Triangle`, `Star`, `Circle`, and `Diamond` and potentially more custom and complex shapes later. Mission elements should be able to be placed in any order, and then choose the number of time steps for calculation.

Input: Unique `DroneID`, mission type (shape), target positions (meters), Drone Position (`x, y, z`)
Output: Mission confirmations and steps, coordination between drone tasks
Contribution: This functionality allows for coordination between drones and drone fleets and moving them to their target position.

F3 — Path Planning

Description: This will calculate the most efficient and optimal path for the drone. It should find the shortest path that does not collide with another drone or obstacle. Drones must be able to stop and have collision avoidance through reversing the velocity.

Input: Target position (`x, y, z`), Potential obstacles
Output: Waypoints / trajectory, collision warnings, collision avoidance
Contribution: Ensures drones arrive at intended position with the most efficient path while avoiding all collisions.

F4 — Simulation & Visualization

Description: The `SimulationEngine` class will handle the visualization update drone positions, mission progress, and show all of this graphically.

Input: Time step (s), Drone commands, drone positions, mission plan
Output: Updated positions at time steps, show graphical representation of drones, paths, and environment
Contribution: Provides a visualization to show the drone movements and display the overall drone show.

3 Object-Oriented Programming (OOP) Design

3.1 Class Relationship Overview

The system consists of **four primary classes** and one supporting struct. The design follows the *single-responsibility principle*: each class encapsulates exactly one aspect of the simulation.

3.2 OOP Design Table

Class Name	Attribute / Method	Access	Data Type	Description
	droneID	private	int	Unique ID number for drone
	position	private	struct	Current position of drone
	velocity	private	float	Current velocity of drone
	updatePosition()	public	void	Updates the position of drone
	detectCollision(otherDrone)	public	bool	Will tell if drone will collide with another drone
	addPosition()	public	void	Add a new position for the drone
	getPosition()	public	void	Get the current position of the drone
	getPattern()	public	void	Get the pattern the drone is doing
	getDroneID()	public	int	Get the ID number of drone
	setColor()	public	void	Set the color of the drone LED
	drone[]	private	int	Array of drones in fleet
	droneAngle	private	double	Angle drones are flying in
FleetManager	droneDistance[]	private	double	Array of drone distances from fleet
	dronePosition[]	private	double	Position of drones from fleet
	addDrone(droneID)	public	void	Adds a drone to the fleet

Class Name	Attribute / Method	Access	Data Type	Description
	deleteDrone(droneID)	public	void	Deletes a drone from the fleet
	missionID	private	int	Unique ID for mission element
	missionType	private	string	Information about mission
MissionPlanner	planPath(droneID, target)	public	vector	Tell a drone where to go
	assignMission(Missio:	public	void	Assign a group of drones to a mission
	getMissionPlan()	public	void	Get the current mission of a group of drones
	deleteMission(missio:	public	void	Delete a mission from the queue
	addMission()	public	void	Add a mission to the queue
	timeStep	private	float	How long mission element will take
SimulationEngine	runStep()	public	void	Runs the mission element
	displayStatus()	public	void	Displays graphics and status of fleet

3.3 Key OOP Requirements

- **≥ 4 Classes:** `Drone`, `FleetManager`, `MissionPlanner`, `SimulationEngine` — each with a distinct, single responsibility.
- **Private vs. Public Attributes:** All data members are `private`; external access is only through public methods.
- **Encapsulation:** Hidden, private attributes that should be protected, logic structures, and array managements
- **Scope resolution:** Control class ownership and reach through `::` and `this->`

4 System Overview — Input, Process, Output

4.1 High-Level Description

This simulator follows a **Input** → **Process** → **Output** model. Inputs are given by the user to configure drones, fleets, and the mission given to them. The process executes as a user defined time-stepped loop. Outputs are computed and printed in the terminal or displayed in the visualization engine.

4.2 IPO Diagram

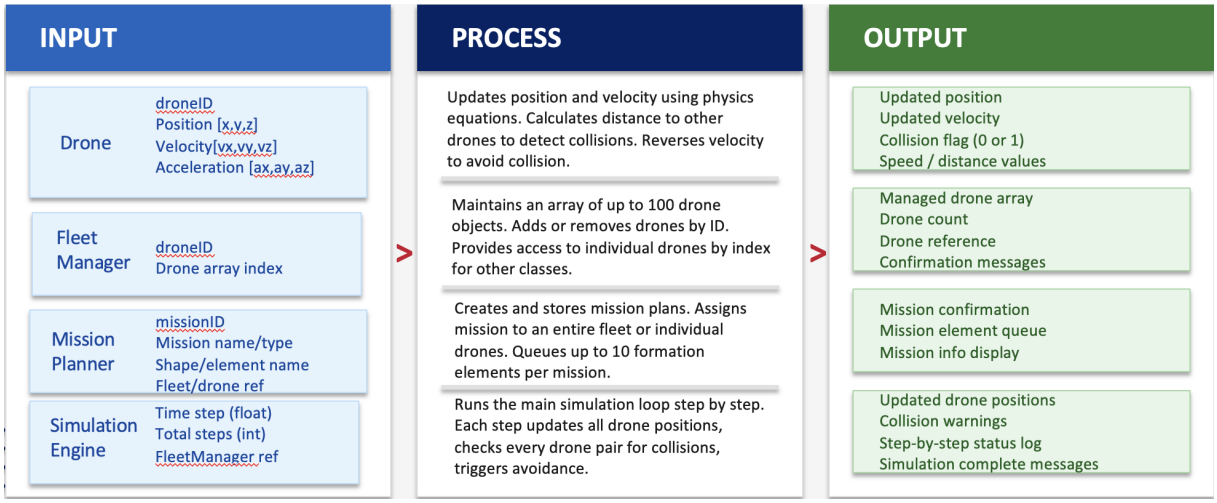

Figure 1: Input-Output Diagram

4.3 Detailed Step Descriptions

4.3.1 Input

→ **Fleet Configuration** How many drones in the fleet and how many used in each shape? Some shapes require a minimum number of drones, such as 3 for a triangle.

→ **Starting Positions:** The starting position of each drone in relation to a chosen origin.

→ **Number of steps:** How many time steps (s) should the simulation take and show the position for, and how long will it take.

→ **Mission Elements:** What shapes will be incorporated into the drone show and in what order?

4.3.2 Process

At each discrete time step t (inputted resolution):

Step 1. **Generate Starting conditions for drones.** Either inputted or default to 0 for position, velocity, and acceleration. However, not all drones can start at 0 as that will cause a collision warning.

Step 2. **Mission Elements.** Select the mission elements and order for the drone show. Current selection includes a circle, triangle, diamond and star.

Step 3. **Route Calculation.** Using physics equations such as $distance = \sqrt{X_{Drone1}^2 + X_{Drone2}^2}$ and $x(t) = x_0 + v_0 t + \frac{1}{2}at^2$ to calculate where the drone needs to go and the steps at times along that path.

Step 4. **Collision Detection.** Drones will compare routes to see if any will intersect within a given distance. If two drones may collide, then it will display a collision warning.

Step 5. **Display Route.** Using either a visualizer or the terminal display, the positions of each drone will be displayed at the inputted time steps.

4.3.3 Output

→ **Positions of drones:** The position in meters will be displayed at each of the time steps. The velocity and the acceleration at that time can also be calculated.

→ **Collision Flags:** If two drones will come within a defined distance of each other, then it will display a collision warning and prevent the mission elements from proceeding.

→ **Status Log:** Display in the terminal the full status of drones along the time steps.

→ **Visualization/simulation:** Visualization of the drone show with a UI to input mission elements and then show the drones moving.

5 Condensed Pseudo Code

Drone class

```
Class Drone:
    Private:
        droneID, position[3], velocity[3], acceleration[3], color

    Constructor(id, x, y, z, vx, vy, vz, ax, ay, az):
        set all values

    Default Constructor:
        set all values to 0

    Setters: setDroneID, setPosition, setVelocity, setAcceleration,
    setColor
    Getters: getDroneID, getPosition, getVelocity, getAcceleration,
    getColor

    getSpeed:
        return sqrt($vx^2$ + $vy^2$ + $vz^2$)

    getDistanceFromOther(otherDrone):
        calculate dx, dy, dz
        return sqrt($dx^2$ + $dy^2$ + $dz^2$)

    updatePosition(time):
        position += velocity * time

    updateVelocity(ax, ay, az, time):
        velocity += acceleration * time

    stop:
        set velocity to [0, 0, 0]

    reset:
        set position, velocity, acceleration to [0, 0, 0]

    detectCollision(otherDrone, collisionDistance):
        if distance < collisionDistance:
            return 1 (collision)
        return 0 (no collision)

    avoidCollision(otherDrone, collisionDistance):
        if detectCollision == 1:
            reverse velocity

    stats.accumulate()
  END FOR

  OUTPUT  stats.printReport()
END PROGRAM
```

Fleet Manager class

```
Class FleetManager:
    Private:
        drone[100], droneCount, droneAngle

    Constructor:
        droneCount = 0, droneAngle = 0
    addDrone(droneID):
        if droneID already exists:
            print "already exists"
            return
        add drone to array
        droneCount++
    deleteDrone(droneID):
        find drone with droneID
        shift all drones after it left by one
        droneCount--
    getDrone(index):
        return drone[index]
    getDroneCount:
        return droneCount
    displayFleet:
        for each drone:
            call drone.displayInfo()
```

Mission Planner class

```
Class MissionPlanner:
    Private:
        missionID, missionName, missionType
        missionRunNames[10], missionRunCount

    Constructor(ID, name, type):
        set all values, missionRunCount = 0
    Default Constructor:
        set defaults, missionRunCount = 0
    Setters: setMissionID, setMissionName, setMissionType
    Getters: getMissionID, getMissionName, getMissionType
    displayMissionInfo:
        print missionID, missionName, missionType
    assignMission(fleet):
        print mission assigned to fleet
    assignMission(drone):
        print mission assigned to drone
    addMissionElement(name):
        if missionRunCount < 10:
            add name to missionRunNames
            missionRunCount++
        else:
            print "limit reached"
    deleteMission:
        reset all values to defaults
```

SimulationEngine class

```
Class SimulationEngine:
    Private:
        timeStep, totalSteps, currentStep, fleet

    Constructor(timeStep, totalSteps, fleet):
        set all values, currentStep = 0

    runStep:
        if timeStep <= 0: print error, return
        currentStep++
        for each drone in fleet:
            call updatePosition(timeStep)
        for each pair of drones (i, j):
            if detectCollision(drone[i], drone[j]):
                print warning
                call avoidCollision
        call displayStatus()

    runFullSimulation:
        if totalSteps <= 0: print error, return
        while currentStep < totalSteps:
            call runStep()
        print "simulation complete"

    displayStatus:
        print currentStep, totalSteps, droneCount
        for each drone:
            print droneID and position
```

6 C++ Implementation Highlights

SimulationEngine displaying collision detection functions - Core C++ Implementation

```
for (int i = 0; i < fleet.getDroneCount(); i++) {
        for (int j = i + 1; j < fleet.getDroneCount(); j++) {
            if (fleet.getDrone(i).detectCollision(fleet.getDrone(j), 1.0)) {
                cout << "Warning: Collision detected between Drone "
                     << fleet.getDrone(i).getID() << " and Drone "
                     << fleet.getDrone(j).getID() << endl;
            }
        }
    }

void SimulationEngine::displayStatus() {
    cout << "Current Step: " << currentStep << "/" << totalSteps << endl;
    cout << "Drones in fleet: " << fleet.getDroneCount() << endl;
    for (int i = 0; i < fleet.getDroneCount(); i++) {
        double* pos = fleet.getDrone(i).getPosition();
        cout << "Drone " << fleet.getDrone(i).getID()
             << " - Position: (" << pos[0] << ", " << pos[1] << ", " << pos[2] << ")" << endl;
    }
```

Key Design Decisions

- **D1.** **Four separate classes** to control each aspect of the program and ensure different parts work together and are accessible.
- **D2.** **Integrating UI** Connecting the UI and graphics to the backend and making it run correctly and show the correct graphics using AI help.
- **D3.** **Collision Detection:** Creating functions in order to efficiently calculate the distance between drones and deciding what distance (based on the size of the drones) to implement anti collision measures
- **D4.** **Use of pointers for `position`, `velocity`, and `acceleration` arrays** Using pointers to efficiently use memory and find position elements.

7 Challenges, Solutions & Lessons Learned

7.1 Challenges & Solutions

#	Challenge	Solution
1	Coordinating code files; scope issues between different class methods	Used scope resolution operators as well as `this->` pointers to clarify specific objects
2	Uploading to GitHub and merge conflicts	Coordinated files before pushing and edited where necessary to ensure a final version can be pushed
3	Out of scope errors between classes	Used scope resolution operators and `this->` operator
4	Memory leaks from position pointers.	Used valgrind to detect leaks and then fixed them with `delete`

7.2 Lessons Learned

- **OOP Class design** creates reuseable and widely applicable code, making the overall program much more efficient
- **Checking for memory leaks** is incredibly valuable for working code, and using smart pointers can make this even easier.
- **Choosing data structures** can be very important, such as making a `array` rather than a vector `vector` depending on the application.
- **Documenting bugs** is very helpful for later looking back for what went wrong if the same error occurs again.
- **GitHub organization** is very important to easily find files and prevent merge conflicts

8 Limitations & Future Work

Current Limitation	Proposed Future Extension
Simple Shapes Only	Include more options for shapes pre-coded or allow users to upload their own images to be rendered as droens
Only simulation drone usage	Implement the code with real drones and allow it to communicate with a real fleet of drones
Only 2D shapes	Add more of a 3D element with 3D figures or rotation of elements in space
Only 100 drones allowed	Allow a varying number of drones per fleet, up to a reasonable number
Only 4 drone classes	Implement more classes to handle more aspects of the show
Only 10 mission elements allowed	Allow varying numbers of mission elements, and more flexibility with their implementation and timing.

9 GitHub Repository Structure

Repository Layout

```
DroneFleetSimulator/
|
|-- README.md
|-- docs/
| |-- System_Design_Overview.pdf
|-- pseudocode/
| |-- pseudocode.txt
|-- src/
| |-- main.cpp
| |-- Drone.h
| |-- Drone.cpp
| |-- FleetManager.h
| |-- FleetManager.cpp
| |-- MissionPlanner.h
| |-- MissionPlanner.cpp
| |-- SimulationEngine.h
| |-- SimulationEngine.cpp
```

A Sample Program Output

Expected Console Output - End-of-Run Report

```
===== Smart Traffic Intersection Simulator =====
  Duration      : 3600 sec
  Arrival Rate  : 8.0 vehicles/min/lane
  Green Phase   : 30 sec  |  Yellow: 5 sec  |  Red: 25 sec
-------------------------------------------------
  LANE 0  Throughput: 1820  AvgWait:  18.4 s  MaxQ: 12
  LANE 1  Throughput: 1795  AvgWait:  19.1 s  MaxQ: 14
  LANE 2  Throughput: 1810  AvgWait:  18.7 s  MaxQ: 11
  LANE 3  Throughput: 1832  AvgWait:  17.9 s  MaxQ: 13
-------------------------------------------------
  TOTAL Throughput  :  7257 vehicles
  GLOBAL Avg Wait   :  18.5 seconds
  Vehicles Dropped  :   143  (1.9%)
  Phase Efficiency  :  87.3%
=================================================
```